

Code Explanation

Transaction Analytics Pipeline Code Explanation

The code is for a transaction analytics pipeline that loads transaction data, cleans and enriches it, calculates various metrics, and saves the results to a specified output path. It uses Apache Spark for data processing and is designed to be run from the command line with optional arguments for customizing the input and output paths.

Overview

The transaction analytics pipeline is a Python script that utilizes Apache Spark for data processing. It is designed to be flexible, allowing users to specify input and output paths via command-line arguments. The pipeline performs several key tasks: loading transaction, customer, and store data; cleaning and enriching the transaction data; calculating customer, category, and regional metrics; identifying top customers; and saving the processed data to a specified output location.

Step 1: Initialize Spark Session and Set Default Paths

This step initializes a Spark session using the `get_spark_session` function and sets default paths for the transaction data, customer data, store data, and output if they are not provided.

```
spark = get_spark_session()

if not date:
    date = (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")
if not transaction_path:
    transaction_path = f"/data/transactions/date={date}"
if not customer_path:
    customer_path = "/data/reference/customers"
if not store_path:
    store_path = "/data/reference/stores"
if not output_path:
    output_path = f"/data/analytics/date={date}"
```

Step 2: Load Transaction, Customer, and Store Data

This step loads the transaction data, customer data, and store data into DataFrames using the `load_transactions`, `load_customer_data`, and `load_store_data` functions, respectively.

```
try:
    transactions_df = load_transactions(spark, transaction_path)
    customer_df = load_customer_data(spark, customer_path)
    store_df = load_store_data(spark, store_path)
    print(f"Loaded {transactions_df.count()} transactions")
except Exception as e:
    print(f"Error loading data: {str(e)}")
    raise
```

Step 3: Clean and Enrich Transaction Data

This step cleans the transaction data using the `clean_transaction_data` function and then enriches it with customer and store information using the `enrich_transaction_data` function.

```
clean_df = clean_transaction_data(transactions_df)
enriched_df = enrich_transaction_data(clean_df, customer_df, store_df)
```

Step 4: Calculate Metrics

This step calculates various metrics, including customer metrics, category metrics, and regional metrics, using the `calculate_customer_metrics`, `calculate_category_metrics`, and `calculate_regional_metrics` functions, respectively. It also identifies the top customers based on the customer metrics.

```
customer_metrics = calculate_customer_metrics(enriched_df)
category_metrics = calculate_category_metrics(enriched_df)
regional_metrics = calculate_regional_metrics(enriched_df)
top_customers = identify_top_customers(customer_metrics)
```

Step 5: Save Processed Data

This step saves the enriched transaction data, customer metrics, category metrics, regional metrics, and top customers to the specified output path in Delta format.

```
enriched_df.write.mode("overwrite").format("delta").save(f"{output_path}/enriched_transactions")
customer_metrics.write.mode("overwrite").format("delta").save(f"{output_path}/customer_metrics")
category_metrics.write.mode("overwrite").format("delta").save(f"{output_path}/category_metrics")
regional_metrics.write.mode("overwrite").format("delta").save(f"{output_path}/regional_metrics")
top_customers.write.mode("overwrite").format("delta").save(f"{output_path}/top_customers")
```

Step 6: Parse Command-Line Arguments and Run the Pipeline

This step uses the `argparse` library to parse command-line arguments for the date, transaction path, customer path, store path, and output path, and then runs the `run_pipeline` function with these arguments.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Run transaction analytics pipeline")
    parser.add_argument("--date", type=str, help="Processing date (YYYY-MM-DD)")
    parser.add_argument("--transaction-path", type=str, help="Path to transaction data")
    parser.add_argument("--customer-path", type=str, help="Path to customer data")
    parser.add_argument("--store-path", type=str, help="Path to store data")
    parser.add_argument("--output-path", type=str, help="Base path for output data")

    args = parser.parse_args()

    run_pipeline(
```

```
date=args.date,  
transaction_path=args.transaction_path,  
customer_path=args.customer_path,  
store_path=args.store_path,  
output_path=args.output_path  
)
```